Paper 1 and Stage 1 Reconstruction

A study guide for Sahil and Partner

Our project asks whether an image can be recovered when only one quarter of its Fourier coefficients are available. Stage 1 explores this with two simple baselines on handwritten digits. Paper 1 motivates using a learned image generator as a richer prior in later stages.

What Paper 1 proposes

Bora, Jalal, Price, and Dimakis present Compressed Sensing using Generative Models at ICML 2017. They recover images from fewer linear measurements than pixels by using a trained generator to restrict the possible answers. [1]

A generator G converts a short latent vector z into an image. Recovery searches for a z whose generated image agrees with the measurements. The experiments use a VAE for MNIST and a GAN for faces; Figure 1 compares reconstruction error as the measurement count changes. [1]

The main assumption is that the true image lies close to something the generator can represent. The theory also needs suitable sensing matrices and sufficiently good optimization. Its Gaussian-measurement guarantees do not automatically cover our Cartesian Fourier masks. [1]

A missing feature can remain missing if the generator cannot express it. The paper separates representation, measurement, and optimization error. These explain why additional measurements or a lower measurement loss may not completely fix a reconstruction. [1]

How the search works

The observed data are y, the unknown image is x, the sensing operator is A, and η represents measurement noise.

$$y = Ax + \eta$$

In the authors' recovery code, pretrained generator weights stay fixed while the optimizer updates the latent vector. Each step generates an image, predicts its measurements, and reduces the mismatch. Different initial latent vectors provide multiple restarts. A latent penalty can favor ordinary codes. The best candidate is selected by the recovery objective, without using the true image. [2]

$$loss(z) = \| AG(z) - y \|^2 + \lambda \| z \|^2$$

The first term measures disagreement with acquired data. The second discourages large latent codes; λ sets its strength. The reference image is used afterward to evaluate accuracy.

Why a prior changes the answer

Imagine the only measurement is a + b = 1. Both (1, 0) and (0.5, 0.5) satisfy it. A model restricted to equal-valued pairs selects (0.5, 0.5). That choice fails if the true pair was (1, 0). A prior resolves ambiguity by restricting the possible answers, so its restrictions must fit the image.

What Stage 1 is about

Stage 1 builds a small, working inverse problem before adding a nonlinear generator. It uses native 28 × 28 MNIST digits with 784 pixel values. The notebook computes their Fourier transforms and retains 7 of 28 complete lines, including 3 central lines: 196 of 784 complex coefficients, or 25%. Measurements are noiseless and simulated. [3]

$$y = MFx$$

F is the Fourier transform and M keeps the sampled coefficients. Removing Fourier coefficients removes frequency information across the image; it does not simply black out individual pixels. The lost information can produce blur and repeated structures after inversion.

Zero filling

Place the acquired complex coefficients in a full Fourier array, set missing entries to zero, and apply the paired inverse FFT. This baseline uses the acquired data without learning what digits look like. For the real-valued MNIST task, the notebook evaluates the real part after inversion and retains the complex result in its saved arrays.

$$x_{ZF} = \Re\left(F^{-1} y\right)$$

Here y is understood as the full Fourier array with zeros at unmeasured locations. Both real and imaginary components are retained until the inverse transform is complete.

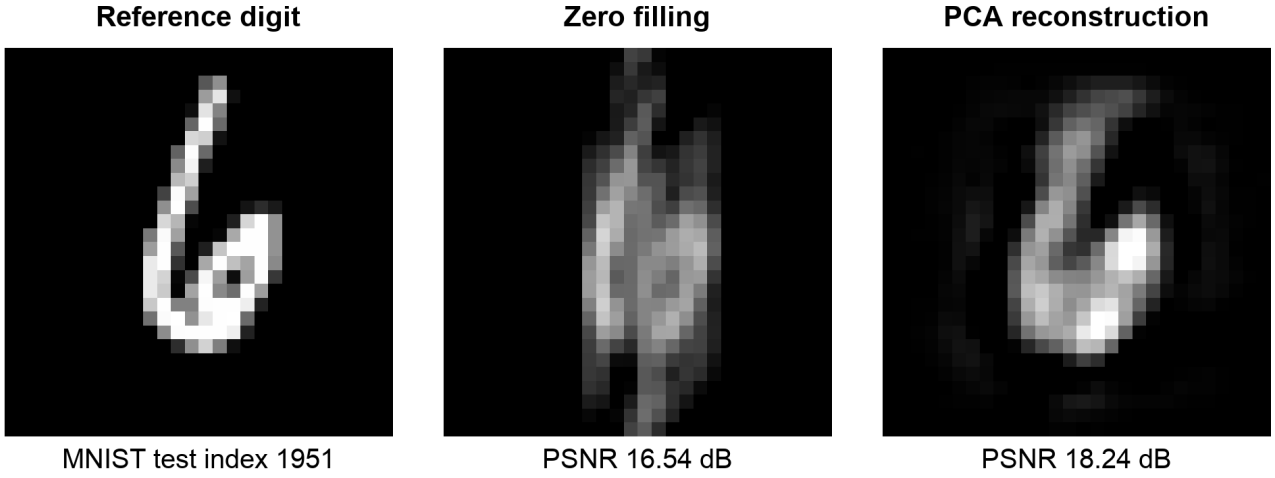
The principal subspace prior

PCA learns the average training digit μ and directions U describing common variations. It represents a candidate image as the average plus a weighted combination of those directions. The weights c are the unknowns during reconstruction.

$$x \approx \mu + Uc$$

Recover c by matching the candidate's sampled Fourier coefficients to the observed values. The ridge penalty limits large coefficients. The solver keeps both parts of the complex residual even though c is real.

$$\min_c \| MF(\mu + Uc) - y \|^2 + \lambda \| c \|^2$$



A fixed held-out digit from the saved run. Display limits are identical; PSNR uses the raw real reconstructions. These are example-image scores, not the test-set means. [4]

How the notebook establishes a result

The run uses 2,000 training images, 200 validation images drawn separately from MNIST's training pool, and 200 official test images. PCA is fitted only on training images. Validation chooses the rank and ridge weight; the test examples are then used for the final comparison. The selected rank is 32 and the ridge weight is 0.01. [4]

Understanding PSNR

MSE is the mean squared pixel error against the reference. PSNR expresses that error on a logarithmic scale; larger values indicate a closer match when the intensity range is fixed. With images scaled to [0, 1], the formula is

$$PSNR = 10 \log_{10}\left(\frac{1}{MSE}\right)$$

The notebook uses the same data range and no per-image rescaling for both methods. A higher average PSNR does not mean every digit is improved, and a good pixel score alone cannot establish preservation of a small feature.

Measured evidence from the saved CPU run

Measurement	Saved result
Mean zero-filling PSNR	15.817 dB over 200 images
Mean PCA PSNR	17.374 dB over 200 images
Eight-image training MSE	6.184e-02 → 3.345e-11
Automatic gradient	-4.041961359738
Finite-difference gradient	-4.041961359746
Gradient relative error	2.204e-12
One PCA fitting run	0.017 seconds
Eight-image training run	0.180 seconds for 1,500 full-batch epochs

These are the recorded measurements from run 20261017T194144_806900Z on an arm64 macOS CPU. They are a reference example; rerun on the machine intended for the project to report its timing. [4]

What the required checks establish

The eight-image check trains a small linear autoencoder on exactly eight training examples. Its rank is sufficient to fit those centered images. The falling loss checks the optimization path; it does not measure generalization. This diagnostic is separate from PCA's one-shot fit.

The gradient check compares automatic differentiation with a centered finite difference: nudge one coefficient up and down, then estimate the change in loss. It uses float64 and a step of 10^{-5}. Agreement supports the local derivative calculation, not the validity of the prior.

The Fourier checks verify a forward/inverse round-trip, a constant image's DC coefficient, an off-center impulse's phase, and forward/adjoint consistency. They target shifts, normalization, and lost complex information that might otherwise leave plausible-looking images.

What to submit for Stage 1

Stage 1 is due October 20. It needs to run and provide evidence of correctness; a weak reconstruction can still meet the assignment. Prepare these items from the actual run. [3]

1. One task/result figure showing a reference digit, the measurement mask, zero-filling, and PCA, with PSNR values. The notebook's main figure also includes error maps and the training curve.
2. The eight-image training-loss curve showing the loss approaching zero. Report its initial and final values.
3. One automatic gradient and its finite-difference counterpart. Include both numbers and identify the checked coefficient and finite-difference step.
4. Measured elapsed time for one training run on the machine you will use. Identify the model and data size; keep training time separate from reconstruction and plotting time.
5. Exactly two sentences about what you expect to be hardest. Sahil and Partner should each contribute their own reasoning.

What Partner should do

Open stage1_partner.ipynb using the shared environment in environment.yml, restart the kernel, and run all cells. The numerical code is supplied. Complete the six comment-only response cells with data notes, the measurement explanation, PSNR interpretation, training-check interpretation, gradient/timing evidence, and the caption and difficulty sentences.

The finished handoff is the annotated partner notebook, its saved figure and results, and an individual contribution entry. Review it together: Partner should be able to explain the comparison and the limits of each check, while Sahil checks the formulation and numerical conclusions.

How this connects to the later stages

The PCA model μ + Uc is a linear generator. Later, a VAE decoder replaces it with a nonlinear generator G(z), while measurements still constrain recovery. The project then uses a score prior with posterior sampling. Stages 3–4 move to the specified MedMNIST dataset and include the required artificial-lesion experiment with no-insertion controls. That experiment tests whether a method keeps, blurs, removes, or invents small features. [3]

Questions both people should be able to answer

- Why can several images agree with the same incomplete measurements?
- What does PCA learn during training, and what is optimized for a new digit?
- Why can a low measurement error still accompany a wrong image?
- Why does fitting eight training images not prove good test reconstruction?

Sources and files to open

[1] Ashish Bora, Ajil Jalal, Eric Price, and Alexandros G. Dimakis. Compressed Sensing using Generative Models. ICML 2017. Read Sections 1–2, 5.1, and 6; Figure 1(a) compares error with measurement count.

[2] Authors' MNIST reconstruction implementation, especially vae_estimator. It restores a trained model and optimizes the latent variable.

[3] Local course requirements: tasks.md and COURSE_UPDATES.md. Project procedure and responsibilities: PROJECT_OUTLINE.md.

[4] Local implementation: stage1_mnist.ipynb and stage1_partner.ipynb. Numerical source: outputs/stage1/20261017T194144_806900Z/summary.json and run_data.npz.